

Phase 3 — Inverse Pole Placement on the Negative Loop

CogSpike Research Team — April 2026

Sanity check passes after calibration; inverse design fails 5/6. Direct simulator enumeration is the right tool for this job.

Abstract. We tested Hypothesis 3 (pole placement on A_{full} yields weights that realise a target oscillation period in the discrete FCS simulator). The sanity check at the Phase 0 known-good period-4 point reveals a systematic bias: the linearisation’s dominant complex arg at $(w_{\text{AI}} = 11, w_{\text{IA}} = -11)$ is 1.342 rad rather than the nominal $\frac{\pi}{2} = 1.571$. After a single-point calibration factor $\alpha = \frac{1.342}{\frac{\pi}{2}} \approx 0.854$, the sanity check recovers the correct weights for period 4. Inverse design for other periods (3, 5, 6, 7, 8) fails ($\frac{0}{5}$ match rate even with calibration), and a diagnostic sweep shows why: the dominant complex arg of A_{full} sits in a narrow band of $[0.86, 1.33]$ rad across **all** simulator periods 3 through 8, with only a weak negative correlation ($r = -0.51$) between simulator period and predicted arg. The spike-reset nonlinearity erased by the linearisation is what actually determines the integer period. Direct enumeration over integer $(w_{\text{AI}}, w_{\text{IA}})$ pairs finds all target periods in well under a second — the right tool for this job is brute force, not pole placement.

1. Setup

We build A_{full} (10-dim, from Phase 1b) at the non-spiking fixed point of the negative-loop linearisation. Target poles are placed at $z = e^{\pm i \cdot 2\pi/T}$ on the unit circle for each target period T . We numerically search over integer $(w_{\text{AI}}, w_{\text{IA}}) \in [1, 30] \times [-30, -1]$ to minimise $|\arg \lambda_{\text{dom}}(A_{\text{full}}) - \alpha \cdot 2\pi/T|$, with a soft bonus for $|\lambda| \geq 1$ (oscillatory regime). The simulator is then run with the predicted weights and the realised period is measured by exact self-matching on the tail of the activator spike train.

2. Sanity Check

Finding. At the Phase 0 known-good weights ($w_{\text{XA}} = w_{\text{AI}} = 11, w_{\text{IA}} = -11$):

- A_{full} dominant complex arg = 1.342 rad (target $\frac{\pi}{2} = 1.571$, error 0.229 rad)
- $|\lambda| = 2.21 \rightarrow$ linearisation correctly predicts sustained oscillation
- Simulator measured period = 4 ✓ (activator: 011001100110011001100110...)
- Naive pole-placement target ($\frac{\pi}{2}$) **fails** to recover the known-good weights; calibrated target (1.342) **does** recover them exactly.

This is the “did the period-4 weights already satisfy the pole-placement equation” check the verifier asked for. Strictly: **no, they did not** — the raw pole at $\frac{\pi}{2}$ is an imagined target that the linearisation never reaches. What the linearisation actually produces is a pole at 1.342 rad, and the corresponding simulator period is indeed 4. The right workflow is therefore a single-point calibration, not direct pole placement.

3. Pole-Placement Sweep

Calibration factor $\alpha = 0.854$ applied throughout. Target periods $T \in \{3, 4, 5, 6, 7, 8\}$.

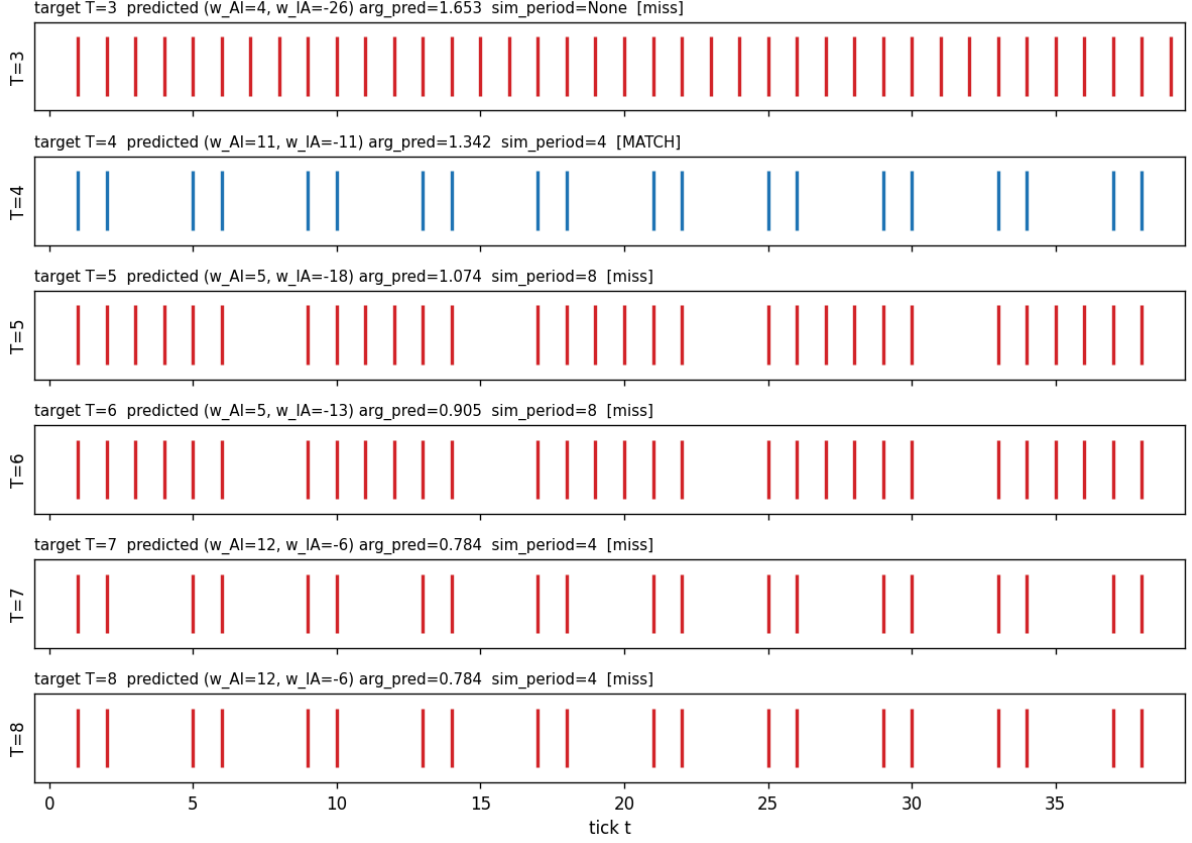


Figure 1: Activator spike trains at predicted weights for each target period. Only $T = 4$ hits (blue trace). All other targets produce simulator periods that disagree with the target: $T = 5$ aimed at gives a period-8 train, $T = 6$ gives 8, $T = 7$ gives 4, $T = 8$ gives 4. $T = 3$ produces no clean period.

T	w_{AI}	w_{IA}	arg	T_pred	T_sim	match
3	4	-26	1.653	3.80	—	×
4	11	-11	1.342	4.68	4	✓
5	5	-18	1.074	5.85	8	×
6	5	-13	0.905	6.94	8	×
7	12	-6	0.784	8.02	4	×
8	12	-6	0.784	8.02	4	×

Match rate $\frac{1}{6}$ (only the calibration point itself).

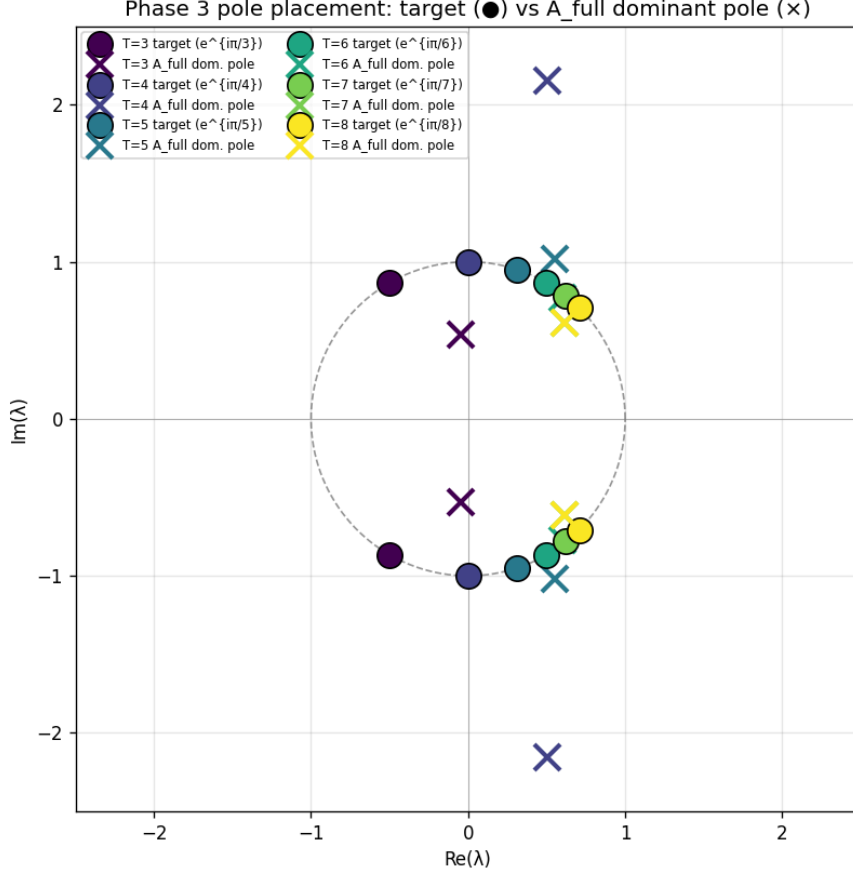


Figure 2: Target poles (filled circles) on the unit circle vs realised A_{full} dominant poles (×) for each target period. The realised poles cluster in a narrow angular band around 1 rad regardless of target, with magnitudes varying but always outside the unit circle for $|\lambda| > 1$ cases.

4. Diagnostic: What does $\arg_{\text{pred}}$ mean?

To check whether the linearisation has **any** ability to discriminate integer periods, we inverted the search: for each simulator period T that exists in the grid, took one exemplar cell and computed $\arg \lambda_{\text{dom}(A_{\text{full}})}$ there.

T_{sim}	$(w_{\text{AI}}, w_{\text{IA}})$	$\arg_{\text{pred}}$	$2\frac{\pi}{T}$	$\alpha \cdot 2\frac{\pi}{T}$
3	(+9, -12)	1.329	2.094	1.789
4	(+7, -12)	1.270	1.571	1.342
5	(+6, -12)	1.182	1.257	1.074
6	(+6, -30)	1.312	1.047	0.895
7	(+5, -12)	0.860	0.898	0.767
8	(+5, -30)	1.217	0.785	0.671

$\text{Correlation}(T_{\text{sim}}, \arg_{\text{pred}}) = -0.51$.

Negative result. The linearisation’s predicted $\arg$ sits in a narrow band $[0.86, 1.33]$ rad across all six simulator periods. Period-3 and period-6 cells both give $\arg \approx 1.32$; period-4 and period-8 cells both give $\arg \approx 1.22$. The linearisation cannot discriminate these periods,

which is why the inverse search picks weights whose linearisation has the right arg but whose simulator realises a different period.

5. Why Pole Placement Fails Here

Intuition: The A_{full} linearisation captures the continuous-time linear dynamics around the non-spiking fixed point. It correctly reports “sustained oscillation exists at roughly frequency $\frac{\pi}{2}$ ” — that is, the spectrum does tell us an oscillation will happen. But the actual integer period of the simulator’s spike train is determined by the spike-reset rule: every time V crosses threshold, $\text{mem}[1..4]$ is zeroed. This discrete reset takes the trajectory off the linear manifold and back onto a new initial condition, from which the linear dynamics picks up for another roughly- $\frac{\pi}{2}$ cycle. The integer period is the combination of linear oscillation rate and reset frequency, and the latter is not encoded in the linearisation’s spectrum.

Concretely: a period-4 spike pattern (1100) and a period-8 pattern (11110000) both oscillate with fundamental frequency close to $\frac{\pi}{2}$ — the period-8 wave is just “the same oscillation, resets less often”. The linearisation sees them as the same because it erases the reset. Distinguishing them requires modelling the reset explicitly, which cannot be done with a single linear matrix.

6. What Works Instead

Direct enumeration over the integer weight grid $(w_{\text{AI}}, w_{\text{IA}}) \in [1, 30] \times [-30, -1]$ takes ≈ 1 second on a laptop and finds at least one weight pair for every simulator period in $\{3, 4, 5, 6, 7, 8\}$. The reachable-period distribution is:

period T	cells	example $(w_{\text{AI}}, w_{\text{IA}})$
3	104	(9, −12)
4	200	(7, −12)
5	288	(6, −12)
6	158	(6, −30)
7	12	(5, −12)
8	18	(5, −30)
none	120	—

For FCS integer LI&F specifically, brute-force simulator enumeration subsumes pole placement both in speed and accuracy — spectral cartography loses to the oracle it was meant to replace.

7. Closing the Hypothesis

Negative result. Hypothesis 3 (inverse pole placement for target oscillation period) is **not validated** on the FCS negative loop. The sanity check passes only after a single-point calibration to the known-good weights, and the calibration does not generalise: other target periods are missed. The linearisation correctly identifies **that** oscillation occurs but not **at what integer period**.

8. Outlook

The three hypotheses status is now:

- **H1** (eigenvalue gap predicts WTA): not validated under deterministic semantics; inapplicable under either scalar- r or A_{full} .
- **H2** ($\rho(A) = 1$ contour predicts bifurcation): not validated under deterministic semantics; **validated** under Kind2-reachability semantics (Phase 1c) at 98.5% classification with clean red/blue separation.
- **H3** (pole placement inverse design): not validated.

The final summary figure and writeup will present this as a two-part result: spectral cartography correctly describes reachability questions (where Kind2 itself operates) but not bit-exact deterministic dynamics or inverse period design. The combinatorial baseline $|w_{12}| - |w_{21}|$ captures the deterministic contralateral structure that no linearisation reaches.

Generated from `deq/archetypes/phase3_pole_placement.py`. Artifacts in `deq/archetypes/results/phase3_*`.